

4. Binary Tree Programming

Write a method `sumLeaves` to be added to the `IntTree` class from class (see cheat sheet). Your method should return the sum of the data stored in all of the leaf nodes. Your method should not alter the tree.

4. Binary Trees

```
public int sumLeaves() {
    return sumLeaves(overallRoot);
}

private int sumLeaves(IntTreeNode node) {
    if (node != null) {
        if (node.left == null && node.right == null) {
            return node.data;
        } else {
            return sumLeaves(node.left) + sumLeaves(node.right);
        }
    }
    return 0;
}
```

5. Linked Lists

Write a method `minToFront` to be added to the `LinkedList` class. Your method should move the smallest element value to the front of the linked list. Suppose a `LinkedList` variable named `list` stores these elements:

[7, 9, 12, 5, 3, 17]

If you made the call of `list.minToFront()`, the list would then store the elements:

[3, 7, 9, 12, 5, 17]

If the list has more than one occurrence of the minimum value, the first occurrence is the only one that should be moved. For example, [7, 9, 3, 12, 5, 3, 17, 3] becomes [3, 7, 9, 12, 5, 3, 17, 3]. If the list is empty, calling your method should have no effect.

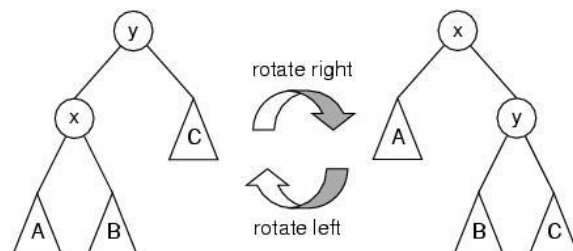
For full credit, obey the following restrictions in your solution. A solution that disobeys them can get partial credit.

- Do not call any other methods on the `LinkedList` object, such as `add`, `remove`, or `size`.
- Do not create new `ListNode` objects (though you may have as many `ListNode` variables as you like).
- Do not use other data structures such as arrays, lists, queues, etc.
- Your solution should run in $O(N)$ time, where N is the number of elements of the linked list.

SORU: A 3-ary max heap is like a binary max heap, but instead of 2 children, nodes have 3 children. Find mapping.

SORU: (20 pts.) We suppose that an AVL tree node has the following data structure.

```
struct node{
    int key;
    struct node *left;
    struct node *right;
    int height;
};
```



Complete the following function that performs right rotation. Note that height member of some nodes should be updated.

```
struct node *rightRotate(struct node *y){  
    .....  
    return ....
```